

Universidad Simón Bolívar
Departamento de Computación y Tecnología de la Información
CI-3661 — Laboratorio de Lenguajes de Programación I
Integrante: Renata Colón – 1810649
Entrega: 22 de Octubre de 2026

Problema 7: Evaluación por sustitución, con tu carnet

Sea d la lista con los cuatro últimos dígitos de tu carnet, en orden. Por ejemplo, para el carnet 19-10437, d = [0, 4, 3, 7]. Escribe a mano o en texto plano, paso a paso como en la clase 3, la:

(a) foldr (-) 0 d hasta llegar a un número.

Respuesta:

d = [0, 6, 4, 9]

```
  foldr (-) 0 d
  = 0 - foldr (-) 0 [6, 4, 9]
  = 0 - (6 - foldr (-) 0 [4, 9])
  = 0 - (6 - (4 - foldr (-) 0 [9]))
  = 0 - (6 - (4 - (9 - foldr (-) 0 [])))
  = 0 - (6 - (4 - (9 - 0)))
  = 0 - (6 - (4 - 9))
  = 0 - (6 + 5)
  = 0 - 11
  = -11
```

(b) foldl (-) 0 d hasta llegar a un número.

Respuesta:

d = [0, 6, 4, 9]

```
  foldl (-) 0 d
  = foldl (-) (0 - 0) [6, 4, 9]
  = foldl (-) ((0 - 0) - 6)  [4, 9]
  = foldl (-) (((0 - 0) - 6) - 4)  [9]
  = foldl (-) ((((0 - 0) - 6) - 4) - 9)  [0]
  = (((0 - 0) - 6) - 4) - 9
  = ((- 6) - 4) - 9
  = (-10) - 9
  = -19
```

(c) Con tu definición de miFoldl, la reducción de miFoldl (-) 0 [d1, d2], donde d1 y d2 son los dos primeros dígitos de d. Cada paso debe corresponder a una ecuación de tu código o a una regla de β-reducción.

Respuesta:

miFoldl: miFoldl f z xs = foldr (\x acc g -> acc (f g x)) id xs z

Tal que nos piden miFoldl (-) 0 [0, 6]

Llamemos step al combinador: \x acc g -> acc (g - x):
Donde en la función de paso step:

- x: elemento actual de la lista.
- acc: acumulador de foldr (la función construida hasta el momento).
- g: el acumulador actual que se va transformando en miFoldl.

1. Sustituimos la definición de miFoldl (-) 0 [0, 6] = foldr step id [0, 6] 0
2. Desenrollamos el primer paso de foldr sobre [0, 6] (donde y = 0 y ys = [6]):
= (step 0 (foldr step id [6])) 0
3. Hacemos β-reducción de step sustituyendo x=0 y acc=(foldr step id [6]):
= (\g -> (foldr step id [6]) (g - 0)) 0
4. Hacemos β-reducción aplicando esta función al argumento g = 0:
= foldr step id [6] (0 - 0)
5. Desenrollamos el segundo paso de foldr sobre [6] (donde y = 6 y ys = []):
= (step 6 (foldr step id [])) (0 - 0)
6. Hacemos β-reducción de step sustituyendo x = 6 y acc = (foldr step id []):
= (\g -> (foldr step id []) (g - 6)) (0 - 0)
7. Hacemos β-reducción aplicando esta función al argumento g = (0 - 0):
= foldr step id [] ((0 - 0) - 6)
8. Caso base de foldr en la lista vacía [] (devuelve id):
= id ((0 - 0) - 6)
9. Aplicamos la definición de id (id k = k):
= (0 - 0) - 6
10. Evaluaciones aritméticas paso a paso:
= 0 - 6
= -6

(d) Explica en un párrafo por qué (a) y (b) dan distinto, y da una condición sobre el operador op y el valor z que garantice que foldr op z xs == foldl op z xs para toda lista finita xs.

Respuesta:

Los resultados obtenidos en (a) y (b) son distintos porque el operador (-) no cumple con la propiedad asociativa (es decir, (a - b) - c != a - (b - c)). Esto influye directamente en la evaluación debido a que foldr agrupa las operaciones asociándolas hacia la derecha, mientras que foldl lo hace hacia la izquierda. Por lo tanto, el orden de los paréntesis altera el resultado aritmético. Para garantizar que foldr op z xs == foldl op z xs para toda lista finita

xs, se debe cumplir que el operador op sea asociativo (cumpliendo (a op b) op c = a op (b op c)) y que el valor z sea el elemento neutro del operador op tanto por la izquierda como por la derecha (cumpliendo x op z = x y z op x = x).